

Nama : Carlos Fadellilah Rama

Kelas : TI-2E

Matkul : Prak PBO job 4

Percobaan 1

1. Kegunaan method setter dan getter:
 - Setter (set...): Untuk mengisi atau mengubah nilai atribut yang berakses private dari luar kelas.
 - Getter (get...): Untuk mengambil atau membaca nilai atribut yang berakses private dari luar kelas.
2. Perbedaan penggunaan konstruktor default dan berparameter:
 - Konstruktor Default: Digunakan saat menginstansiasi objek tanpa langsung memberikan nilai awal pada atributnya (nilai atribut diisi belakangan menggunakan method setter).
 - Konstruktor Berparameter: Digunakan saat menginstansiasi objek sekaligus langsung memberikan nilai awal untuk atribut-atributnya saat objek dibuat.
3. Atribut bertipe object dan baris kode relasi pada class Laptop:
 - Atribut bertipe object: Atribut proc yang bertipe Processor.
 - Baris kode relasi: private Processor proc;.
4. Kegunaan sintaks proc.info():
 - Sintaks tersebut memanggil method info() milik objek proc (Processor). Ini merupakan bentuk delegasi, di mana kelas Laptop menyuruh objek Processor untuk mencetak detail informasinya sendiri.
5. Perbandingan output Langkah 8 dan Langkah 10:
 - Outputnya tidak berbeda (sama persis).
 - Alasan: Pada Langkah 10, instansiasi new Processor("Intel i5", 3) dilakukan secara langsung (*inline/anonymous object*) di dalam argumen konstruktor Laptop, tetapi fungsi dan hasilnya tetap sama dengan menginstansiasinya ke variabel terpisah terlebih dahulu.
6. Jenis relasi (Aggregation/Composition) dan buktinya:
 - Relasi ini termasuk Aggregation.
7. Analisis jika konstruktor diubah menjadi this.proc = new Processor("Generic", 1);:
 - Relasinya berubah menjadi Composition, bukan Aggregation lagi.
 - Alasan: Objek Processor diinstansiasi secara internal di dalam kelas Laptop sendiri menggunakan kata kunci new. Hal ini membuat keberadaan objek Processor terikat penuh pada siklus hidup kelas Laptop dan tidak bisa dimasukkan dari luar.

Percobaan 2

1. Baris program penanda relasi di class Pelanggan: Dua baris deklarasi atribut berikut menunjukkan bahwa kelas Pelanggan memiliki relasi dengan kelas Mobil dan Sopir:
Java
private Mobil mobil;
private Sopir sopir;
2. Alasan perlunya argumen hari pada method hitungBiayaSopir dan hitungBiayaMobil: Objek Mobil dan Sopir hanya menyimpan informasi mengenai tarif atau biaya dasar per hari (atribut biaya), sedangkan durasi penyewaan (hari) dicatat oleh objek Pelanggan. Oleh karena itu, jumlah hari harus dikirimkan dari Pelanggan melalui parameter agar kelas Mobil dan Sopir dapat menghitung total biayanya masing-masing.
3. Kegunaan perintah mobil.hitungBiayaMobil(hari) dan sopir.hitungBiayaSopir(hari): Perintah tersebut digunakan oleh kelas Pelanggan untuk menyuruh (*mendelegasikan*) objek mobil dan sopir agar menghitung biaya sewa berdasarkan durasi hari yang diberikan. Hasil kalkulasi dari kedua method ini kemudian dijumlahkan untuk mendapatkan biaya total.
4. Kegunaan sintaks p.setMobil(m) dan p.setSopir(s):
Sintaks tersebut berfungsi untuk meng-inject atau menghubungkan referensi objek Mobil (m) dan objek Sopir (s) ke dalam atribut objek Pelanggan (p).
5. Kegunaan proses p.hitungBiayaTotal():
Proses ini memanggil method di dalam objek Pelanggan untuk menghitung dan mengembalikan total biaya sewa yang harus dibayar (biaya sewa mobil + biaya sewa sopir).
6. Urutan eksekusi p.getMobil().getMerk():
 - Method p.getMobil() dieksekusi terlebih dahulu dan mengembalikan referensi objek Mobil yang dimiliki oleh pelanggan p.
 - Kemudian, method .getMerk() dipanggil secara langsung pada objek Mobil yang baru saja dikembalikan tersebut untuk mendapatkan nama merk mobilnya.
7. Error jika p.setMobil(m) tidak dipanggil dan alasannya:
 - Error: NullPointerException.
 - Alasan: Sebelum p.setMobil(m) dipanggil, atribut mobil di dalam objek Pelanggan masih bernilai null (belum menunjuk ke objek Mobil manapun di memori). Ketika p.hitungBiayaTotal() dijalankan dan mencoba mengeksekusi mobil.hitungBiayaMobil(hari), Java akan menghentikan program karena kita mencoba memanggil method dari variabel referensi yang masih kosong (null).

Percobaan 3

Kegunaan `this.masinis.info()` dan `this.asisten.info()`: Kedua baris tersebut berfungsi untuk mengambil string informasi detail (NIP dan Nama) dari masing-masing objek Pegawai (masinis dan asisten) dengan cara memanggil method `info()` milik kelas Pegawai.

Output MainPertanyaan sebelum diperbaiki dan alasannya:

- **Hasil Output:** Program mengalami kecelakaan saat runtime (*crash*) dan menampilkan pesan error `java.lang.NullPointerException`.
- **Alasan:** MainPertanyaan menggunakan konstruktor 3-parameter yang tidak mengisi nilai atribut asisten. Ketika method `info()` dipanggil, program berusaha mengeksekusi `this.asisten.info()`, padahal atribut asisten masih bernilai `null`.

Isi variabel asisten sebelum guard clause ditambahkan: Variabel asisten berisi `null`. Hal ini terjadi karena variabel referensi objek yang tidak diinisialisasi atau tidak diisi saat pemanggilan konstruktor secara otomatis bernilai `null` di memori.

Kebutuhan pengecekan guard clause pada objek masinis:

- **Tidak perlu.**
- **Alasan:** Pada kedua konstruktor di kelas `KeretaApi` (baik yang 3-parameter maupun 4-parameter), atribut `masinis` **wajib** dimasukkan sebagai parameter. Dengan demikian, nilai `masinis` tidak akan bernilai `null` selama pembuatan objek `KeretaApi` dilakukan sesuai aturan konstruktornya.

Jumlah objek Pegawai yang dibuat:

- Konstruktor tersebut membuat **dua objek Pegawai yang berbeda**.
- **Penjelasan berdasarkan kode:** Pada `MainPercobaan3.java`, kata kunci `new` dipanggil sebanyak dua kali secara terpisah:
- Java

```
Pegawai masinis = new Pegawai("1234", "Spongebob Squarepants"); // Objek 1
```

```
Pegawai asisten = new Pegawai("4567", "Patrick Star"); // Objek 2
```

-

- Kedua objek tersebut menempati lokasi memori yang berbeda sebelum dikirimkan ke dalam kelas `KeretaApi`.

Percobaan 4

1. **Jumlah kursi dalam Gerbong A:** Jumlah kursi adalah **10 kursi** (nomor 1 sampai 10). Hal ini ditentukan saat instansiasi `new Gerbong("A", 10)`; di mana angka 10 dikirimkan sebagai ukuran array `arrayKursi`.
PDF+ 1
2. **Maksud kode `if (this.penumpang != null) { .. }` pada class **Kursi**:** Kode tersebut merupakan *guard clause* yang berfungsi untuk mengecek apakah suatu kursi sudah diisi oleh penumpang atau masih kosong. Jika kursi belum berpenumpang (penumpang bernilai `null`), kode di dalam blok `if` tidak akan dijalankan sehingga menghindari terjadinya error `NullPointerException`.
PDF+ 1
3. **Alasan nilai nomor dikurangi 1 pada method `setPenumpang()` di class **Gerbong**:** Karena indeks *array* dalam Java dimulai dari **0** (*zero-based indexing*). Ketika pengguna memasukkan nomor kursi 1, sistem harus mengakses indeks array ke-0 ($1 - 1 = 0$).
PDF+ 1
4. **Analisis saat menempa kursi penumpang dengan objek baru budi:**
 - **Hasil:** Objek budi akan berhasil menggantikan Mr. Krab pada kursi nomor 1.
PDF
 - **Peringatan Java:** Java **tidak memberi peringatan/error** apapun. Referensi objek penumpang lama pada kursi tersebut secara otomatis tertimpa oleh referensi objek budi yang baru.
PDF+ 1
5. **Modifikasi agar tidak diperkenankan menduduki kursi yang sudah terisi:** Ubah method `setPenumpang()` pada file `Gerbong.java` menjadi seperti berikut:

Java

```
public void setPenumpang(Penumpang penumpang, int nomor) {  
    // Cek apakah kursi pada nomor tersebut sudah ada penumpangnya  
    if (this.arrayKursi[nomor - 1].getPenumpang() != null) {  
        System.out.println("Gagal: Kursi nomor " + nomor + " sudah ditempati!");  
    } else {  
        this.arrayKursi[nomor - 1].setPenumpang(penumpang);  
    }  
}  
}```[cite: 1]
```

6. **Pemilihan antara array vs atribut bernama satu-satu:**
 - **Atribut bernama satu-satu:** Dipilih ketika jumlah objek bagian (*part*) sudah pasti/tetap dan masing-masing memiliki **peran (role) yang berbeda dan**

spesifik (contoh: masinis dan asisten pada Kereta Api, atau proc pada Laptop).
PDF

- **Array:** Dipilih ketika jumlah objek bagian bersifat dinamis, berjumlah banyak, dan **perannya seragam/identik** tanpa perlu nama khusus untuk tiap unitnya (contoh: kumpulan Kursi dalam Gerbong).
PDF

7. Penentuan jenis relasi (**Aggregation vs Composition**) pada Percobaan 4:

- **Gerbong – Kursi (Composition):**

- **Bukti Kode:** Kelas Gerbong membuat sendiri seluruh objek Kursi di dalam method `initKursi()` menggunakan kata kunci `new Kursi(...)` yang dipanggil di dalam konstruktornya:

PDF

- Java

```
this.arrayKursi[i] = new Kursi(String.valueOf(i + 1));  
```[cite: 1]
```

- **Kursi – Penumpang (Aggregation):**

- **Bukti Kode:** Objek Penumpang dibuat di luar (pada `MainPercobaan4`) lalu dimasukkan ke dalam Kursi menggunakan method setter:

PDF

- Java

```
public void setPenumpang(Penumpang penumpang) {
 this.penumpang = penumpang;
}
```

■